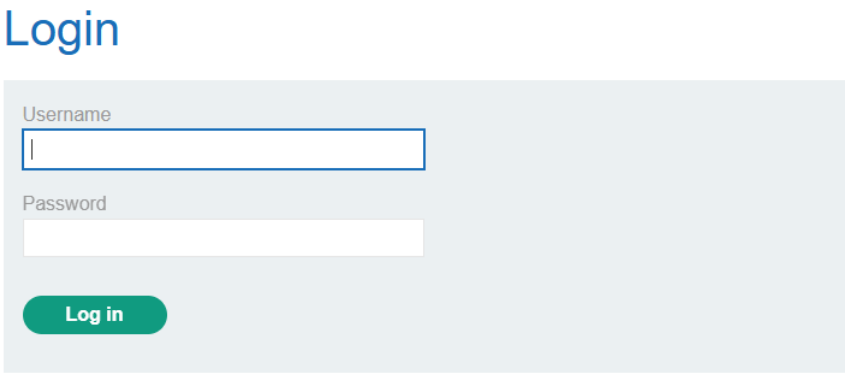
Authentication - klasa napada

Sergej Mika

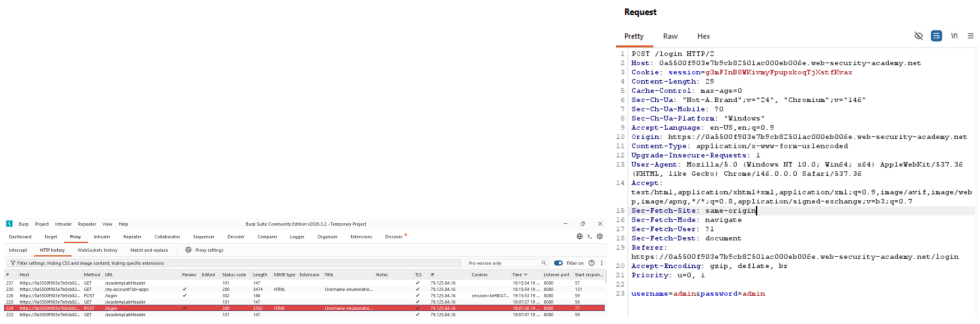
20. april 2026.

1. Lab: Username enumeration via different responses

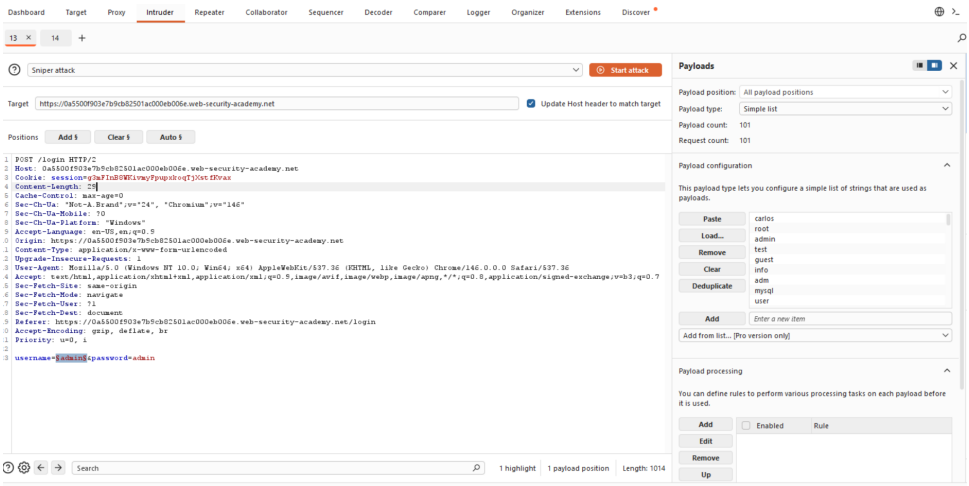
Ovaj lab ima ranjivost username enumeration-a i password brute-force attack-a. Zbog toga naš postupak će izgledati kao u nastavku:



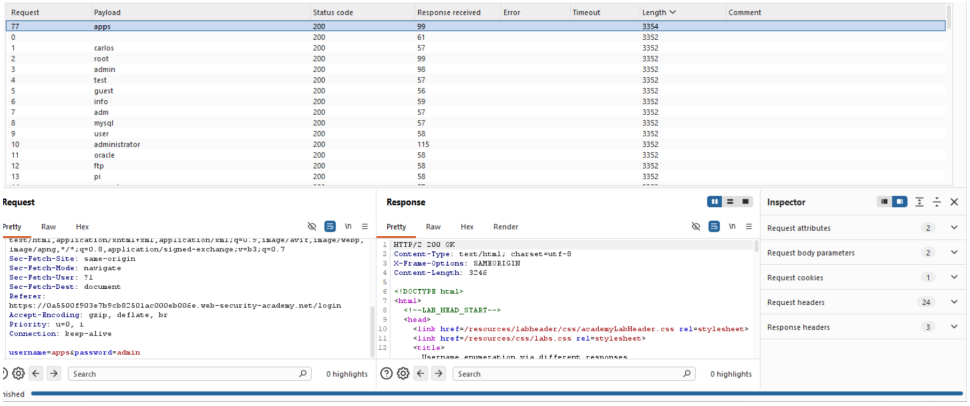
Slika 1: Popunjavamo podatke na login stranici kako bi u Burp Suite dobili oblik našeg requesta



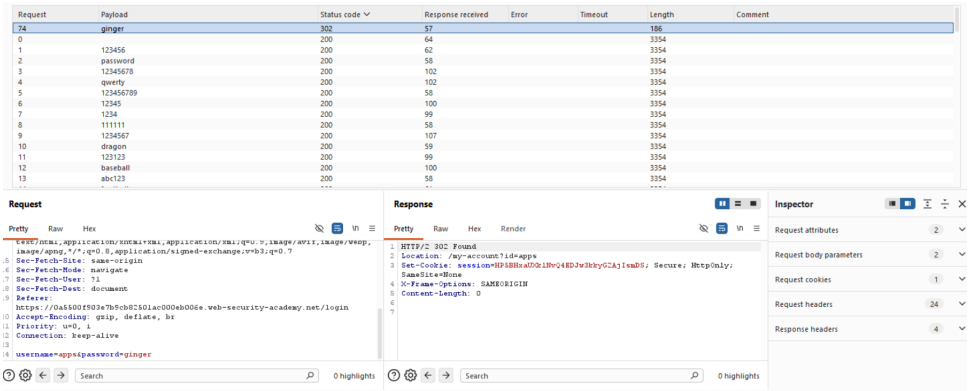
Slika 2: Izgled strukture našeg requesta



Slika 3: Pošto imamo skup username-ova od kojih je neki ispravan. Utvrđivanje koji username je ispravan je moguć pošto u svom response-u server otkriva da li smo uneli invalid username ili invalid password. Samim tim ćemo na osnovu dužine responsa otkriti da li je reč o invalid username-u ili password-u. Kako bi napad izvršili, potrebno je da parametrizujemo username i da mu pasujemo listu vrednosti za request-ove, nakon ćega otpočinjemo napad.



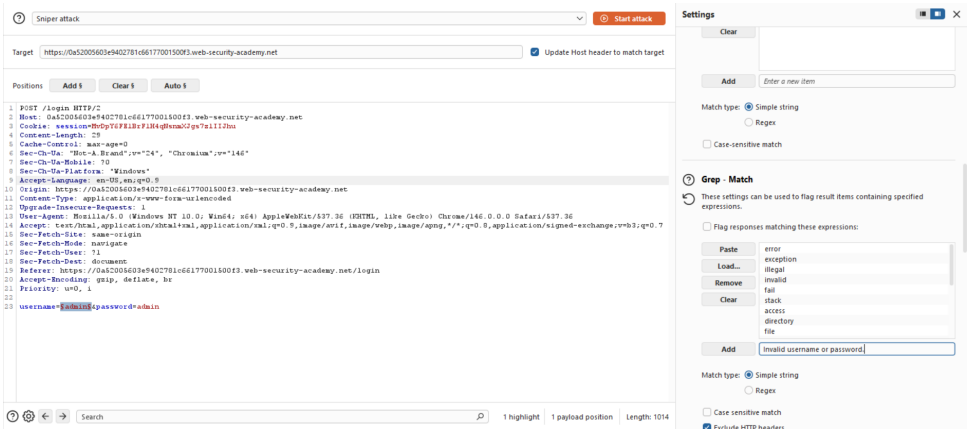
Slika 4: Nakon izvršenog napada i upoređivanja dužina response-ova dobijamo validan username (u ovom slučaju apps).



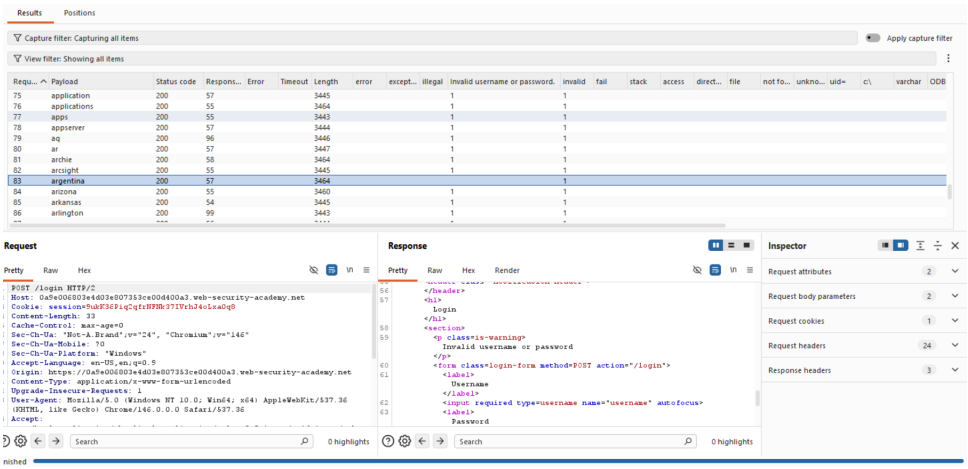
Slika 5: Pošto imamo listu password-a od kojih je jedan validan, kreiramo opet napad tako što će nam username biti postavljen na validan username, a password će biti parametrizovan, pa ćemo proslediti listu mogućih vrednosti. Nakon napada dobićemo jedan response koji vraća kod 302, taj response sadrži validan password jer to znači da smo redirektovani. Sa otkrivanjem validnog username i njemu odgovarajućeg password-a možemo uspešno da se ulogujemo.

2. Lab: Username enumeration via subtly different responses

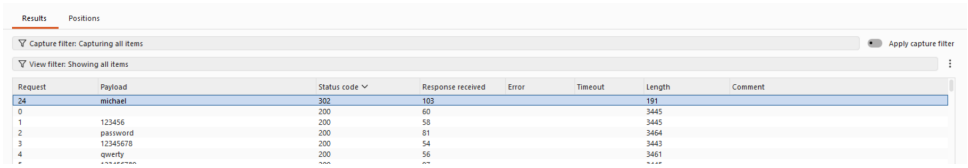
Ovaj lab se razlikuje od prethodnog po tome što ovoga puta nisu svi responsovi iste dužine. Kako bi došli do username enumeration-a moramo koristiti pretragu. Postavićemo u pretragu rečenicu koju dobijemo za neuspešno unet username ili password. Prećićemo kroz listu svih dostupnih username-ova i videti da li dolazi do neke razlike u poruci koju dobijemo.



Slika 6: Podešavanje pretrage da vidi da li će svaki response za neki username da vrati istu poruku.



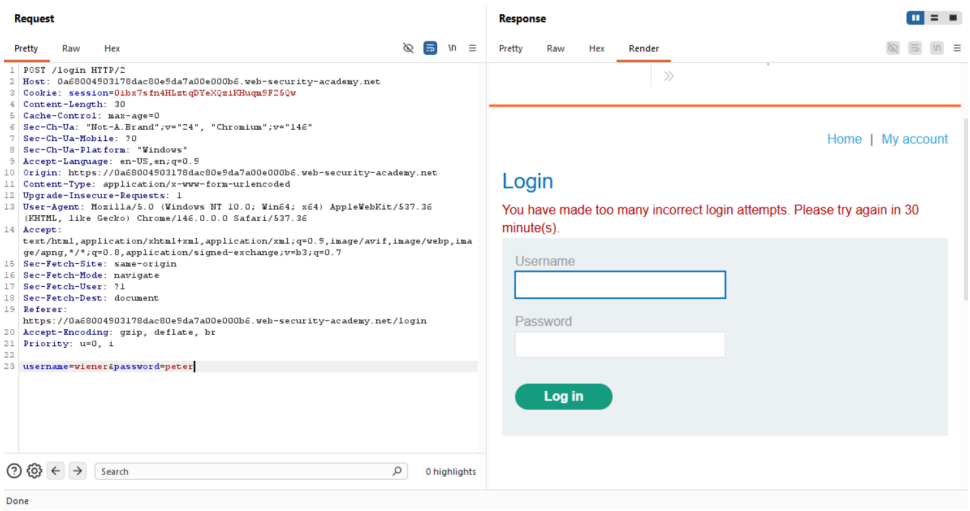
Slika 7: Svi sem jednog responsa su vratili da sadrže datu poruku. Kada uđemo u response koji ne sadrži poruku vidimo da je razlika samo u tome što fali tačka, došlo je do nekonzistentnosti. Developeri su napravili propust.



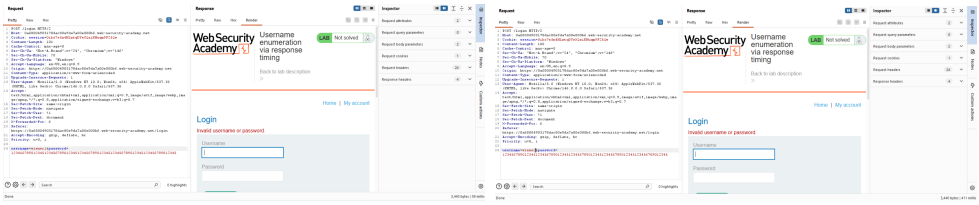
Slika 8: Nakon uspešnog username enumeration-a ponovimo postupak kao u prošlom zadatku, brute force attack-a na osnovu datih password-a. Pronađena odgovarajuća šifra postojećem username-u se može koristiti za login.

3. Lab: Username enumeration via response timing

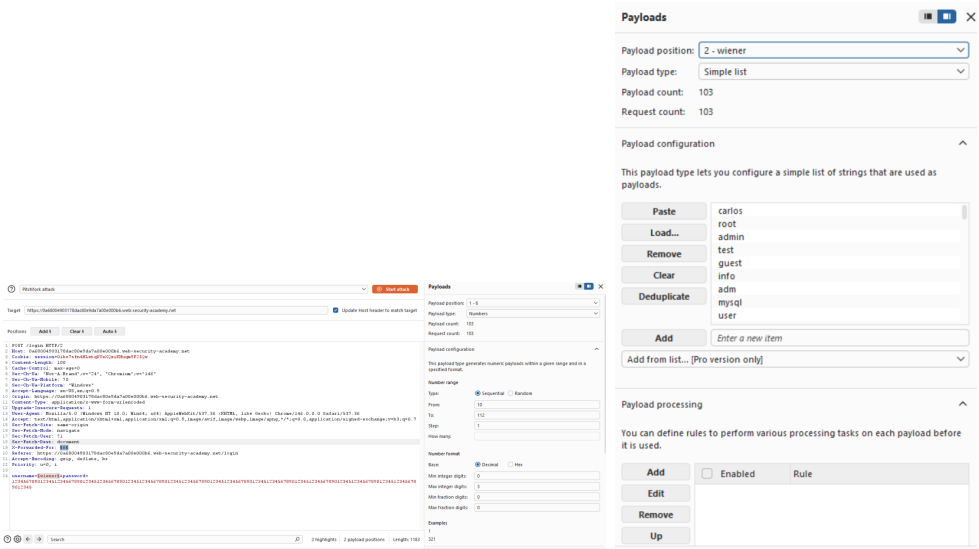
Zadatak sadrži slabu odbranu od brute force napada na osnovu IP adrese.



Slika 9: Ako pokušamo da se ulogujemo neuspešno 3 ili 4 puta, naša IP adresa biva blokirana. Ovaj problem rešavamo dodavanjem header field-a X-Forwarded-For pošto je odbrana od brute force-a najjednostavnijeg oblika (ne koristi proxy proveru). Dodavanjem tog header name-a i pridodavanjem njemu odgovarajući header value koji ćemo menjati svaki request kako bi simulirali promenu IP adrese (možemo lako parametrizovati request u Intruder-u).



Slika 10: Sada želimo da proverimo razliku između vremena koji je potreban da se obradi request sa tačnim username-om, a netačnom šifrom i netačnog username-a. U početku ako stavimo kratku šifru razlika nije toliko velika, ali kako povećavamo dužinu šifre tako se sve više vidi razlika između response time-a request-a sa postojećim username-om i nepostojećim username-om.



Slika 11: U Intruder-u pravimo Pitchfork attack kako bi mogli da imamo dva parametra koji se menjaju po indeksima (prva vrednost iz liste vrednosti prvog parametra sa prvom vrednosti iz liste vrednosti drugog parametra itd...). Kreiramo dva parametra, number za ip adrese, listu mogućih username-ova za username parametar.

Request	Payload 1	Payload 2	Status code	Response receiv...	Respo...
102	111	wiener	200	466	466
7	16	adm	200	445	445

Slika 12: Kako jedina dva koja imaju veliko vreme izvršavanja su wiener koji smo sigurni da je postojeći username i adm koji ćemo pokušati da iskoristimo sada da uradimo brute force napad pomoću liste mogućih password-a. Ponovićemo napad samo sada umesto parametrizacije username-ova i IP adresa, radimo parametrizaciju password-a i IP adresa.

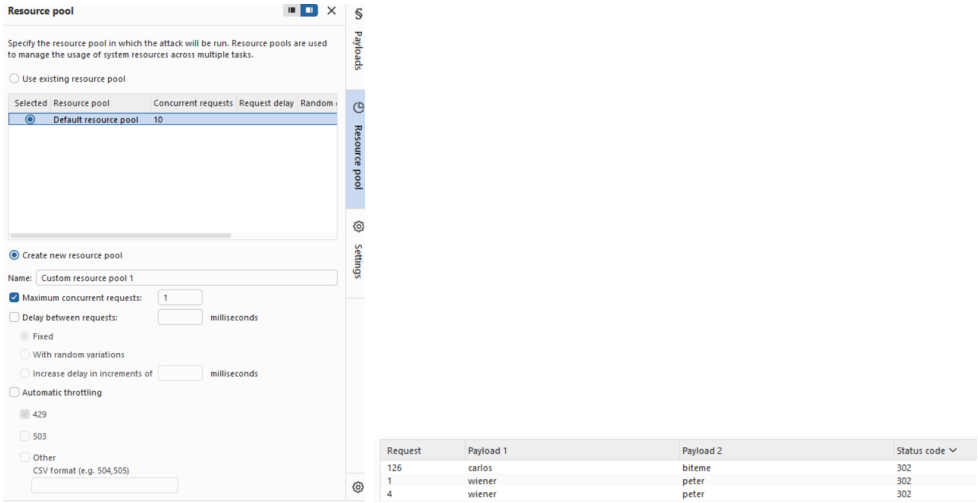
Request	Payload 1	Payload 2	Status code	Response received
1	116	123456	302	57
0			200	435

Slika 13: Dobili smo password 123456 koji je vratio CODE 302 tako da smo uspešno izvršili username enumeration za username adm i onda pronašli njemu odgovarajući password brute force napadom.

4. Lab: Username enumeration via response timing

U ovom zadatku više ne možemo da koristimo vulnerability X-Forwaded-For header field-a. Server trackuje našu pravu IP adresu. Pošto ovaj primer ima vulnerability da pri svakom uspešnom login-ovanju counter za neuspele pokušaje jedne IP adrese se resetuje. Isprobavajući primere u Repeater-u došli smo do zaključka da svaki treći neuspešan login će nas blokirati od pokušaja ponovnog login-a na minut, ali ako se uspešno ulogujemo nakon svaka dva neuspešna pokušaja counter se resetuje i možemo pokušavati koliko god želimo. Pošto imamo postojećeg korisnika (njegov username i password) i username korisnika u čiji nalog želimo da provalimo, izmešaćemo parametar username tako da svaki su prva 2 pokušaja username korisnika kojem želimo da saznamo šifru, a potom ide username korisnika kojem znamo šifru. Takođe šifre će pratiti ovaj redosled, pa u listi

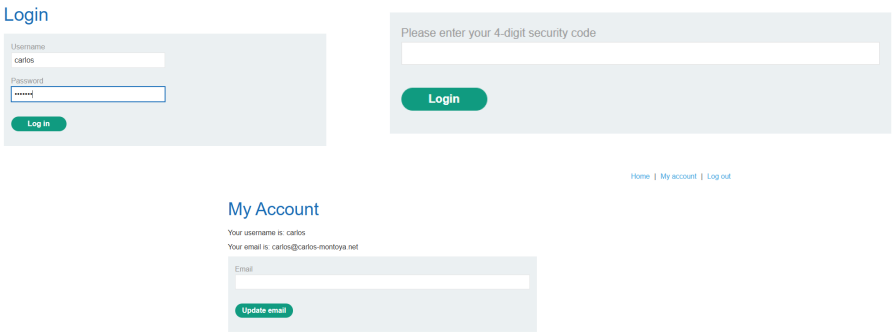
mogućih šifara korisnika na svaku treću poziciju dodajemo šifru korisnika kojem znamo i username i password. Moramo postaviti da Resource Pool radi serijalizaciju, odnosno da ne šaljemo više od jednog request-a u isto vreme, već pre slanja svakog request-a sačekamo response tog requesta kako ne bi došli u situaciju da su tri requesta za neuspešan login bila pre ijednog uspešnog request-a.



Slika 14: Samo jedan response ima CODE 302 za username kojem smo želeli da saznamo password i taj request upravo sadrži password koji nama treba.

5. Lab: 2FA simple bypass

U ovom zadatku ćemo pokušati da exploit-ujemo vulnerability koji često 2FA autentication-je poseduju. To je da u dosta slučajeva kada ukucamo tačan username i password smo već ulogovani bez da smo ukucali code za 2FA, pa možemo samo preći na naš account bez ikakvog kucanja 2FA code-a.



Slika 15: Ukucamo username i password korisnika koje smo nabavili bez da imamo pristup njihovom uređaju za 2FA. Nakon login-ovanja prikazaće nam se polje za unos 2FA code-a. Mi ćemo ovo preskočiti tako što ćemo manuelno ukucati url od home-a. Odatle prelazimo na stranicu account-a i vidimo da smo ulogovani bez ikakvog kucanja 2FA code-a.

6. O klasi napada

Autentifikacija predstavlja glavni stub odbrane svake veb aplikacije. Kada ovaj mehanizam zakaže, posledice su kritične jer napadač direktno preuzima identitet legitimnog korisnika, čime sve ostale bezbednosne kontrole (poput autorizacije) postaju beskorisne.

6.1 Opis klase napada

Napadi na autentifikaciju fokusiraju se na kompromitovanje načina na koji aplikacija potvrđuje da je korisnik upravo onaj za koga se izdaje. Ova klasa napada obuhvata širok spektar tehnika: od klasičnog pogađanja lozinki (brute-force), preko eksploatacije logičkih grešaka u dizajnu (kao što je preskakanje 2FA koraka), pa sve do suptilnih curenja informacija putem odgovora servera. Suština ovih napada je da se iskoristi poverenje koje aplikacija ima u unete kredencijale ili u sam proces prijave.

6.2 Exploit impact

Uspešan napad na autentifikaciju obično rezultira potpunim preuzimanjem naloga (*Account Takeover - ATO*). Uticaj zavisi od nivoa privilegija kompromitovanog naloga:

- **Krađa podataka:** Pristup ličnim informacijama, istoriji transakcija ili poverljivim dokumentima korisnika.
- **Eskalacija privilegija:** Ako napadač kompromituje administrativni nalog, dobija potpunu kontrolu nad celim sistemom, bazom podataka i infrastrukturom.
- **Reputaciona šteta:** Gubitak poverenja korisnika i potencijalne pravne kazne zbog curenja podataka.

6.3 Ranjivosti koje omogućavaju napad

Kao što smo videli u prethodnim laboratorijskim vežbama, napadi uspevaju zbog specifičnih propusta u kodu i konfiguraciji:

- **Inkonzistentni odgovori servera:** Različite poruke o grešci ili čak suptilne razlike u interpunkciji omogućavaju napadaču da potvrdi postojanje korisnika u bazi.
- **Vremenske razlike (Timing differences):** Server troši više vremena na obradu validnih korisničkih imena (npr. zbog provere heša lozinke), što otkriva informacije kroz merno vreme odziva.
- **Nedostatak ograničenja (Rate Limiting):** Dozvoljavanje neograničenog broja pokušaja prijave sa jedne IP adrese ili na jedan nalog.
- **Nepotpuna implementacija MFA:** Situacije gde aplikacija generiše sesiju odmah nakon unosa lozinke, tretirajući drugi faktor (2FA) samo kao opcioni korak na frontendu, a ne kao obaveznu validaciju na bekendu.

6.4 Kontramere i prevencija

Da bi se sistem osigurao, potrebno je primeniti višeslojni pristup zaštiti:

1. **Generičke poruke o greškama:** Bez obzira da li je pogrešno korisničko ime ili lozinka, aplikacija uvek treba da vrati identičan odgovor (npr. "Invalid username or password") bez indikacije koji deo kredencijala je netačan.

2. **Sprečavanje vremenske analize:** Implementacija konstantnog vremena obrade zahteva za prijavu, bez obzira na to da li korisnik postoji u bazi, čime se eliminišu *timing* napadi.

3. **Napredni Rate Limiting:**

   - Blokiranje nakon određenog broja neuspešnih pokušaja (npr. 5 pokušaja).
   - Implementacija *exponential backoff* mehanizma (svaki sledeći pokušaj zahteva duže čekanje).
   - Praćenje ne samo po IP adresi, već i po sesiji i korisničkom imenu.

4. **Striktna MFA politika:** Sesija ne sme dobiti status autentifikovane sve dok oba faktora nisu uspešno validirana na serverskoj strani. Pokušaj pristupa bilo kojoj zaštićenoj stranici pre unosa 2FA koda mora rezultirati automatskom odjavom.

5. **Upotreba sigurnih Headera:** Izbegavanje oslanjanja na `X-Forwarded-For` za bezbednosne provere, jer se ovi headeri lako mogu lažirati.